

2vorlesung

November 12, 2025

1 Funktionen und Module in Python

Vorlesung Theoretische Chemie (WiSe 2024/2025)

2 Basics: Funktionen

Funktionen sind der grundlegender Bestandteil eines Programms. Eine Funktion ist eine festgelegte Abfolge von **Operationen**, die mit einer Reihe von Variablen, den **Argumenten**, ausgeführt wird. Die Argumente werden durch **runde Klammern** () festgelegt, die allgemein das Merkmal von Funktionen sind. Ein Beispiel dafür ist die `print()` Funktion, die alles innerhalb der Klammern ausgibt.

```
[ ]: a = 1
      b = 2
      print("Die Zahl", 1, "ist kleiner als", 2)
```

In Python gibt es eine Reihe von **Built-In Funktionen**, die wesentliche Funktionalitäten zur Verfügung stellen. In der Regel ist es aber notwendig, zusätzlich eigene Funktionen zu nutzen.

3 Basics: Funktionen definieren

Der Beginn einer Definition für eine Funktion wird durch das **def** Keyword signalisiert. Die allgemeine Syntax ist

```
def name(argument1, argument2, ...):
    #Code
    return output1, output2, ...
```

```
[ ]: def exponent(x, power):
      """
      Bildet den Exponenten
      Input:
      x: Float
      Output:
      y: Float
      """
      y = x ** power
      return y
```

```
a = 5
b = exponent(a, 2)
print(a, b)
```

<h3>Wichtig</h3>

<p>Die Anzahl der Input und Output Argumente muss exakt eingehalten werden! Eine Funktion kann
Python-Dokumentation
 zur Verfügung.</p>

<h2>Good practice - Funktionen</h2>

Funktionen werden gesammelt am Anfang des Programms definiert und sollten mit Kommentaren versehen werden um die Aufgaben und Argumente aller Funktionen gut zu dokumentieren

```
[ ]: def function_1():
    """
    Diese Funktion tut nichts.
    Input: -
    Output: str
    """
    return "Ich tue nichts"

def function_2():
    """
    Diese Funktion tut ebenfalls nichts
    Input: -
    Output: str
    """
    return "Ich tue noch weniger"
```

Danach folgt der eigentliche Code Block.

```
[ ]: text_1 = function_1()
text_2 = function_2()

print(text_1)
print(text_2)
```

4 Funktionen verzweigen

Funktionen müssen nicht immer gleich funktionieren - in manchen Fällen kann es nützlich sein Ausnahmen zu definieren

4.0.1 Beispiel: Division durch 0

```
[ ]: def division(a, b):  
    """  
    Dividiere a / b  
    Input:  
    a: Integer / Float  
    b: Integer / Float  
    """  
    # Prüfe ob b sehr klein ist  
    if abs(b) < 10e-12:  
        # Return False  
        return False  
    # Ansonsten: Normale Funktion  
    return a / b  
  
div1 = division(10, 2.5)  
div2 = division(10, 0)  
  
print(div1, div2)
```

<h3>Debugging</h3>

<p>Zwischenergebnisse prüfen kann sehr nützlich zum debuggen / zur Fehlersuche sein! </p>

5 Übung 1: Funktionen

5.0.1 Aufgabe: Schreibe zwei Funktion die zwei gleichlange Listen mit Zahlen elementweise addiert und subtrahiert

Hinweise: - Die Aufgabe benötigt **for-loops** - Was passiert wenn die Listen nicht gleich lang sind?
- Resultate müssen ausgegeben werden mit **return** - Eine leere Liste kann mit [] erstellt werden, neue Werte können mit **.append()** hinzugefügt werden

```
[ ]: def addition():  
    # Addiere die zwei Listen hier  
  
def subtraktion():  
    # Subtrahiere die zwei Listen hier  
  
list1 = [4.3, 2.4, 5.3, 1.2, 4.5]  
list2 = [9.4, 9.2, 1.4, 2.2, 7.8]  
  
add_list = addition()  
sub_list = subtraktion()
```

5.0.2 Aufgabe: Der folgende Code kann durch eine Funktion stark gekürzt werden, indem wiederholende Abschnitte durch einen For-loop mit einer Funktion erledigt werden.

Hinweis: - Viele gleiche Werte können in **Listen** zusammengefasst werden

```
[ ]: # Input Daten
m1 = 10      # Masse Probe 1
m2 = 3       # Masse Probe 2
m3 = 12      # Masse Probe 3
m4 = 15      # Masse Probe 4

V1 = 102     # Volumen Lösung 1
V2 = 99      # Volumen Lösung 2
V3 = 109     # Volumen Lösung 3
V4 = 103     # Volumen Lösung 4

molmasse = 99 # g / mol

# Konzentrationsberechnung
def konzentration(gewicht, molmasse, volumen):
    # Berechnet die Konzentration in mol/l
    # gewicht: Stoffmenge in Gramm
    # molmasse: Molekulare Masse in g / mol
    # volumen: Lösungsmittel in ml
    """
    Berechne die Konzentration in mol / l hier
    """

# Berechnung der Stoffmengen
n1 = m1 / molmasse
n2 = m2 / molmasse
n3 = m3 / molmasse
n4 = m4 / molmasse

# Volumen in Litern
V1 = V1 * 1000
V2 = V2 * 1000
V3 = V3 * 1000
V4 = V4 * 1000

# Berechnung der Konzentrationen
c1 = n1 / V1
c2 = n2 / V2
c3 = n3 / V3
c4 = n4 / V4
```

```
# Output
print("Konzentration 1 beträgt", c1)
print("Konzentration 2 beträgt", c2)
print("Konzentration 3 beträgt", c3)
print("Konzentration 4 beträgt", c4)
```

6 Module in Python

7 Was ist ein Python Modul

- **Modul:** Erweiterung der Python Sprache, die spezielle Funktionen / Konstanten bereitstellt
- ständige Neuimplementierung von Basisfunktionen ist nicht notwendig

Bsp. Konstanten und Mathematische Funktionen:

```
[ ]: import numpy as np
print(np.pi)      # Kreiszahl Pi
print(np.sin(np.pi)) # Sinusfunktion
```

8 Relevante Module für Chemie / Wissenschaften

- **NumPy:** mathematische Funktionen, Lineare Algebra, Daten Einlesen
- **Matplotlib:** Graphen erstellen
- **SciPy:** Regression, Optimierungsprobleme, ...
- **Pandas:** Analyse von großen Datensätzen
- ...

9 Installation von Modulen

- nicht alle Module sind in der Installation von Python enthalten
- Installation abhängig von Paketmanager
 - pip (standard Python Paketmanager) `bash` `pip install numpy`
 - conda Anaconda Paketmanager `bash` `conda install numpy`
- Python Editoren wie PyCharm ermöglichen auch das Installieren mit einer grafischen Benutzeroberfläche

10 Einbinden von Python Modulen

Einfacher Import:

```
import modul
```

Import mit alias:

```
import modul as alias      # bevorzugt
```

Import von spezifischen Funktionen/Objekten:

```
from modul import module_function
```

```
[ ]: import numpy
print(numpy.pi)
print(numpy.sin(0))
```

```
[ ]: import numpy as np
print(np.pi)
print(np.sin(0))
```

```
[ ]: from numpy import pi, sin
print(pi)
print(sin(0))
```

<h2>Good practice - Module</h2>

- Importe von Modulen als allererstes im Python Script
- jedes Modul nur einmal importieren

als erstes Module importieren

```
import module1 as m1
```

```
import module2
```

```
from module3 import function_5
```

danach folgen Funktionsdefinitionen

```
def my_function1():
```

```
...
```

```
return ...
```

danach die eigentlichen Berechnungen

```
a = ...
```

```
b = ...
```

```
print(...)
```

11 Numpy

12 Numpy: Überblick

- Numpy: “Numerical Python”
- **Inhalt**
 - Konstanten: Eulersche Zahl, Kreiszahl π , ...
 - Funktionen: trigonometrische Funktionen, exponential und logarithmus Funktionen, numerische Integration, numerische Ableitung, ...
 - **lineare Algebra**: Matrizen (Tensoren), Matrix Operationen (Eigenwert Löser, Matrix Produkt, ...)
 - einlesen von Datensätzen

13 Numpy: Arrays (Matrizen, Tensoren)

- Arrays bezeichnet den Datentyp zum Speichern eines Tensors

Array-Typ	Bezeichnung
0D-Array	Skalar
1D-Array	Vektor
2D-Array	Matrix
3D-Array	Rang-3 Tensor
nD-Array	Rang-n Tensor

- Elemente im Tensor haben einen einzigen Datentypen (z.B. Float oder Integer)

<h3>Brauch ich Arrays auch wenn man als Experimentalchemiker arbeite?</h3>

<p>Ja: Arbeiten mit Datensätzen, z.B. NMR/UV-Vis/IR Spektren mit x und y listen als 1D A

14 Numpy: Erstellen eines Arrays

14.1 aus Listen

```
[ ]: import numpy as np
my_list = [1, 2, 3, 4]
my_array = np.array(my_list)      # erstellt den 1D-Array
print(type(my_list))

my_array = np.array([1, 2, 3, 4]) # auch so möglich
print(my_array, type(my_array))
```

```
[ ]: import numpy as np
my_array = np.array([[1, 2], [3, 4], [5, 6]]) # erstellt eine Matrix
print(my_array)
```

15 Numpy: Erstellen eines Arrays

15.1 mit generierenden Funktionen

```
[ ]: import numpy as np
my_array = np.linspace(-1.0, 2.0, 10) # 1D-Array: (start, stop, elements)
print(my_array)
my_array = np.arange(-1.0, 2.0, 0.4) # 1D-Array: (start, stop, step)
print(my_array)
```

```
[ ]: import numpy as np
my_array = np.zeros((2, 3)) # 2D-Array: (dim1, dim2)
print(my_array)
```

16 Numpy: Python Listen vs Numpy Arrays

```
[ ]: import numpy as np
my_array = np.array([1, 2, 3, 4])
result = my_array * 3 + 1  # Operation wird für jedes Element ausgeführt
print(result)

result = np.sin(my_array)
print(result)
```

```
[ ]: my_list = [1, 2, 3, 4]
result = my_list + my_list  # Mit Listen geht das nicht
print(result)

#my_list = [[1, 2], [3, 4]]
#my_list[0, 1]
```

17 Numpy: Array Indexierung

- Zugreifen auf ein bestimmtes Element in einem Array
- index in **eckigen Klammern**: `numpy_array[index]`
- Simpel für 1D Arrays:

```
[ ]: import numpy as np
my_array = np.array([1, 2, 3])
print(my_array[0]) # erstes Element
print(my_array[1]) # mittleres Element
print(my_array[2]) # letztes Element
print(my_array[-2]) # auch letztes Element
```

Wichtig

Das erste Element in einem Array hat den Index 0.

18 Numpy: Array Indexierung

- Für 2D Arrays: `my_2d_array[index_axis_0, index_axis_1]`

```
[ ]: import numpy as np
my_array = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])
print(my_array[0, 0]) # element links oben
#
print(my_array[2, 2]) # element rechts unten
```



```
print(my_array[1, 0]) # 2. element in der ersten Spalte
```

19 Numpy: Array Indexierung

- Ausschnitt eines Arrays mit mehr als einem Element:

```
my_1D_array[start_index : end_index] # Ausschnitt mit ":" operator
```

```
[ ]: import numpy as np
my_array = np.array([1, 2, 3, 4, 5])
print(my_array[0:2])
print(my_array[2:]) # weglassen eines index bedeutet alles danach
print(my_array[:2]) # oder davor
print(my_array[:])
```

20 Numpy: Array Indexierung

- Ausschnitt eines 2D Arrays mit mehr als einem Element:

```
my_2D_array[start_index_0 : end_index_0, start_index_1 : end_index_1]
```

```
[ ]: import numpy as np
my_array = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])
print(my_array[0:2, 0:2]) # sub-matrix links oben
print(my_array[:, 0])    # erste Spalte
print(my_array[1, :])    # zweite Zeile
```

21 Numpy: Logische Indexierung

- oft gewünscht: Elemente aus einem Array, die eine Bedingung erfüllen
- Bsp. alle Elemente die größer sind als ein Schwellenwert

```
[ ]: import numpy as np
my_array = np.array([1, 2, 3, 4])
mask = [False, False, True, True] # nur die letzten beiden elemente
print(my_array[mask])
```

```
[ ]: import numpy as np
my_array = np.array([1, 2, 3, 4])
mask = (my_array > 2.0) # erstellt liste mit True oder False Werten
print(mask)
print(my_array[mask])
```

```
[ ]: import numpy as np
my_array = np.array([1, 2, 3, 4])
print(my_array[my_array > 2.0]) # alles in einem Ausdruck
```

22 Numpy: Array Dimensionen

- Die Form eines Arrays erhält man mit der `numpy.shape` Funktion
- Es wird eine Tupel mit den Längen der jeweiligen Dimensionen zurückgegeben

```
[ ]: import numpy as np
my_array = np.array([1, 2, 3, 4, 5]) # 1D Array
my_shape = np.shape(my_array)
print(my_shape)
print(f"Der Array hat die Länge {my_shape[0]}.")
```

```
[ ]: import numpy as np
my_array = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])
my_shape = np.shape(my_array)
print(my_shape)
print(f"Der Array hat die Form {my_shape[0]} mal {my_shape[1]}.")
```

23 Numpy: Array aus Datei einlesen

- Mit der Funktion `numpy.loadtxt` ist es möglich Textdateien zu laden

```
my_array = np.loadtxt("my_txt_file.txt")
```

- optionale Argumente helfen beim einlesen

```
# einlesen von komma-separierten text file, nur die ersten beiden Spalten
my_array = np.loadtxt("my_txt_file.csv", delimiter=',', usecols=(0, 1))
```

```
[ ]: import numpy as np
"""
Inhalt der Textdatei test.txt:

1.0 2.0 3.0
4.0 5.0 6.0
7.0 8.0 9.0
"""

my_array = np.loadtxt("test.txt", usecols=(0))
print(my_array)
```

24 Numpy: Funktionen und Arrays

- numpy stellt sehr viele mathematische Funktionen bereit
- Beispiele einiger Funktionen:

```
[ ]: import numpy as np
my_array = np.array([-2, -1, 1, 2])

print(np.sin(my_array))           # trigonometrische Funktionen
print(np.sum(my_array))           # Summe aller Elemente
print(np.abs(my_array))           # absolut Wert aller Elemente
print(np.log(np.abs(my_array)))   # Logarithmus
print(np.amax(my_array))          # Maximum aller Elementen
print(np.amin(my_array))          # Minimum aller Elemente
print(np.argmax(my_array))        # Index des Maximums aller Elementen
print(np.argmin(my_array))        # Index des Minimums aller Elemente
print(np.mean(my_array))          # Mittelwert
print(np.trapz(my_array))         # numerische Integration mit trapez-regel
print(np.dot(my_array, my_array)) # Skalarprodukt
```

25 Numpy: Achtung bei Array Kopien!

```
[ ]: import numpy as np
import copy
a = np.array([1, 1, 1])
b = a
b[0] = 2
print(a, b)
```

Grund: Python speichert alles als Objekte. Mit `b = a` enthält die variable `b` die Referenz zu dem Objekt der Variable `a`, aber kein neues Objekt wird erzeugt.

Workaround:

```
import numpy as np
import copy
b = copy.deepcopy(a)
```

26 Übung 2: Indizierung von Arrays

26.0.1 Aufgabe: Lass dir die jeweiligen gefragten Werte aus dem Array angeben.

```
[ ]: import numpy as np

my_array = np.array([
    [ 1,  2,  3,  4,  5],
    [ 6,  7,  8,  9, 10],
    [11, 12, 13, 14, 15],
```

```

    [16, 17, 18, 19, 20]
])

""" TODO 1: Ausgabe des Elements mit Wert 3 """
result_1 =
print(result_1)

""" TODO 2: Ausgabe des Elements mit Wert 19 """
result_2 =
print(result_2)

""" TODO 3: Ausgabe der ersten Spalte """
result_3 =
print(result_3)

"""
TODO 4: Ausgabe der Sub-Matrix
[[14, 15],
 [19, 20]]
"""
result_4 =
print(result_4)

""" TODO Bonus: Ausgabe aller Werte größer gleich 16 und kleiner 19 """
print()

```

27 Übung 3: Numerische Integration

Das Integral über eine Gaussfunktion ist gleich der Quadratwurzel der Kreiszahl π .

$$\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi}$$

Integriere die Gaussfunktion numerisch und vergleiche das Ergebnis mit der analytischen Lösung. Benutze dazu die Funktion `numpy.trapz()` ([Dokumentation](#)). Bei einer numerischen Integration kann das Integral über den Bereich $(-\infty, +\infty)$ durch ein Integral über ein endliches Intervall $[a, b]$ angenähert werden. Hilfreiche `numpy` Funktionen/Konstanten für diese Aufgabe: `numpy.pi`,

```

[ ]: """TODO: importiere das numpy modul
      mit alias np
"""
import

"""TODO: definiere eine Funktion, die für ein

```

```

        gegebenes x eine Gaussfunktion zurück
        gibt
"""
def gauss_function(x):
    return ____

"""TODO: erstelle einen 1D-Array"""
x = ____
y = gauss_function(x)

"""TODO: numerische Integration"""
numeric_integral = ____

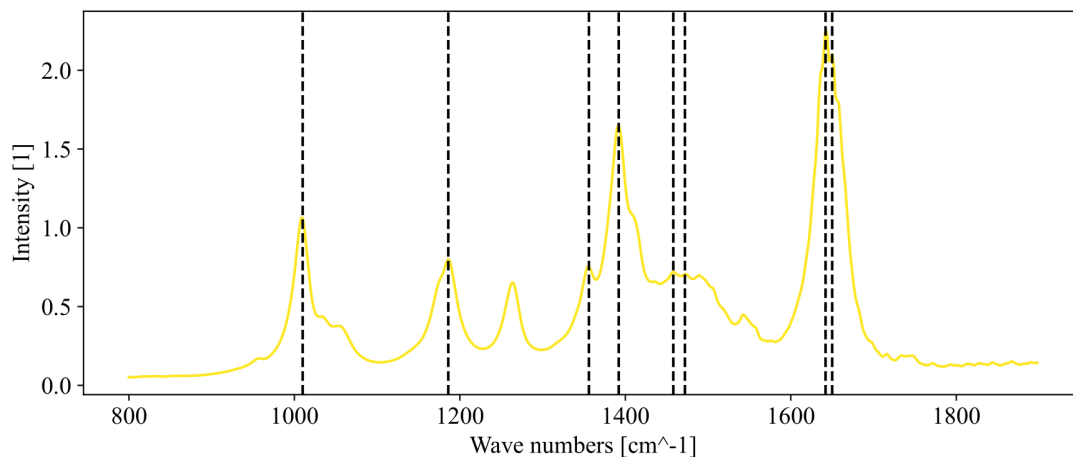
"""TODO: analytische Lösung"""
correct = ____

print(f"Numerisches Integral: {numeric_integral}")
print(f"Analytische Lösung: {correct}")
print(f"Differenz: {numeric_integral - correct}")

```

28 Spektrum analysieren

Für die Übung ist ein IR-Spektrum von Dimethylacetamid (DMAc) gegeben.



###

Übung 4: Daten Laden und auswerten Nutze `numpy.loadtxt()` um das Spektrum aus `dmac.csv` zu laden. Finde anschließend das Absorptionsmaximum und lass dir die Wellenzahl des Maximums ausgeben.

```

[ ]: import numpy as np
      dmac_spectrum = np.loadtxt().T

      print()

```

28.0.1 Finde die Positionen der anderen markierten Peaks in der Abbildung

Lokale Absorptionsmaxima, die keine globalen Maxima sind, können mit Funktionen, die das Maximum aus einer Liste oder einem Array suchen schwer zu finden sein. Da wir aber wissen **wo** die verschiedenen Maxima auftauchen, können wir den Suchbereich eingrenzen um die gewünschten Peaks zu erhalten. Wir möchten die exakten Positionen von 4 Peaks finden:

- Zwischen 900 cm^{-1} und 1100 cm^{-1}
- Zwischen 1100 cm^{-1} und 1300 cm^{-1}
- Zwischen 1300 cm^{-1} und 1500 cm^{-1}
- Zwischen 1600 cm^{-1} und 1800 cm^{-1}

Die Positionen der Peaks kann bestimmt werden, indem zunächst das Spektrum mit der Funktion `mask_spectrum` auf einen Teilbereich eingegrenzt wird, um danach innerhalb dieses Teilbereichs mit `np.argmax()` das Maximum zu finden.

28.0.2 Hinweise:

- List comprehensions können nützlich und effizient sein:

```
some_list = [element for element in Liste if Bedingung ]
```

- Über mehrere Listen kann mit der `zip` Funktion geloopt werden:

```
for el1, el2 in zip(list1, list2):
```

```
[ ]: def mask_spectrum(x, y, x_min, x_max):
    """
    Entfernt den Teil des Spektrums der nicht zwischen x_min und x_max liegt
    x: Liste der Wellenzahlen
    y: Liste der Intensitäten
    x_min: Kleinster X-Wert im Ausschnitt des Spektrums
    x_max: Größter x-Wert im Ausschnitt des Spektrums
    Output:
    x_masked: Wellenzahlen im Bereich x_min bis x_max
    y_masked: Absorption im Bereich x_min bis x_max
    """

    return x_masked, y_masked

'''TODO: Spektrum wie in der vorigen Übung laden und x- bzw. y-Werte_
↪definieren'''

x =
y =

peak_x_min = [900, 1100, 1300, 1600]
peak_x_max = [1100, 1300, 1500, 1800]

for i in :
    print(f"Das Maximum von Peak {i} liegt bei {} cm-1")
```